

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

# Pertemuan 9

## Integrasi AI dengan Gemini

Membuat AI menyarankan kategori, gedung, dan urgensi laporan, sementara keputusan tetap di tangan pelapor

---

**Muhammad Habib Algifari, S.Kom., M.T.I.**

Tahun 2026



Program Studi Teknik Informatika  
Fakultas Teknologi Industri  
Institut Teknologi Sumatera

# Tiga jawaban dari tiket keluar minggu lalu

| Pertanyaan                                           | Jawaban singkat                                                                   |
|------------------------------------------------------|-----------------------------------------------------------------------------------|
| Kenapa kode kamera tidak bisa ditulis di commonMain? | API kamera berbeda per platform dan hanya ada di SDK masing-masing                |
| Apa yang harus dilakukan aplikasi saat izin ditolak? | Tetap berfungsi tanpa fitur itu, jelaskan alasannya, dan beri jalan ke pengaturan |
| Di mana izin dideklarasikan untuk Android dan iOS?   | AndroidManifest.xml dan Info.plist                                                |

# Judul seadanya, admin yang menebak

Laporan seperti di samping masuk setiap hari. Judulnya rusak, kategorinya kosong. Padahal deskripsinya sudah cukup jelas untuk menebak kategori, gedung, dan seberapa mendesak.

## DISKUSI 2 MENIT

Bisakah AI membantu menebak kategori, gedung, dan tingkat urgensi dari deskripsi, tanpa mengambil alih keputusan pengguna?

### Laporan baru

Judul

rusak

**Judul seadanya**

Deskripsi

AC di GKU 1 R201 netes  
terus ke meja dosen, lantainya  
jadi licin

**Informasinya ada di sini**

Kategori

Belum dipilih

**Kosong, admin harus menebak**

# Saat pulang nanti, kalian bisa

1

## Merancang prompt

Menyusun peran, konteks, format, dan contoh, lalu meminta keluaran JSON dengan skema.

2

## Memanggil Gemini

Mengirim permintaan dari KMP dengan Ktor, dengan kunci API yang tidak masuk repository.

3

## Saran yang bisa dikoreksi

Menampilkan hasil AI sebagai saran, menangani error, dan mengukur akurasi.

Capaian ini mendukung **CPMK0503**: menerapkan teknologi sistem cerdas dalam pengembangan perangkat lunak, dalam bentuk fitur AI yang membantu pengguna tanpa mengambil alih keputusannya.

## Bagian 1

# Prompt yang bisa diandalkan

Empat bagian prompt, dan kenapa jawaban bebas tidak bisa dibaca kode.

# Empat bagian prompt

| Peran                                                            | Konteks                                                                         | Format                                                                    | Contoh                                         |
|------------------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------|
| Siapa AI di sini.<br>Kamu membantu admin fasilitas kampus ITERA. | Yang perlu ia tahu.<br>Daftar gedung, daftar kategori, dan arti urgensi Tinggi. | Bentuk jawaban.<br>Hanya JSON dengan field kategori, gedung, dan urgensi. | Satu kasus beserta jawaban yang kita harapkan. |

Deskripsi dari pelapor ditaruh paling akhir. Yang menyusun prompt adalah aplikasi, bukan pelapor.

BANGUN BARENG

# Prompt kita hanya: Tentukan kategori laporan ini: rusak

Apa jawaban AI? Kalau dikirim tiga kali, apakah jawabannya sama? Tulis tebakanmu, baru kita coba di AI Studio.

# Prompt seadanya dan prompt LaporKampus

| Bagian   | Prompt seadanya                   | Prompt LaporKampus                        |
|----------|-----------------------------------|-------------------------------------------|
| Peran    | Tidak ada                         | Admin fasilitas kampus ITERA              |
| Konteks  | Tidak ada                         | Daftar gedung, kategori, dan arti urgensi |
| Format   | Bebas                             | Hanya JSON sesuai skema                   |
| Contoh   | Tidak ada                         | Satu deskripsi dengan jawabannya          |
| Hasilnya | Paragraf yang berbeda setiap kali | Tiga field yang bisa dibaca kode          |

# Prompt disusun oleh kode

commonMain/ai/Prompt.kt

```
fun susunPrompt(deskripsi: String) = buildString {  
    // peran  
    appendLine("Kamu membantu admin fasilitas kampus ITERA.")  
    // konteks  
    appendLine("Gedung: " + GEDUNG.joinToString())  
    appendLine("Kategori: " + KATEGORI.joinToString())  
    appendLine("Urgensi Tinggi bila membahayakan orang.")  
    // format  
    appendLine("Jawab hanya dengan JSON sesuai skema.")  
    // contoh  
    appendLine("Contoh: lampu R305 GKU 2 kedip berarti")  
    appendLine("kategori Listrik, gedung GKU 2, urgensi Rendah.")  
    append("Deskripsi pelapor: $deskripsi")  
}
```

GEDUNG, KATEGORI, dan URGENSI adalah daftar yang sama dengan pilihan di form laporan.

Prompt menjadi fungsi biasa. Bisa dibaca saat code review dan bisa diuji minggu depan.

# Peta hari ini: dari deskripsi ke saran

## Dari deskripsi menjadi saran yang bisa dikoreksi



AI hanya menyarankan. Pelapor mengubah urgensi, lalu memutuskan sendiri.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

## COBA LANGSUNG



Scan atau klik untuk membuka  
tiga latihan hari ini. Semuanya  
jalan tanpa kunci API karena  
memakai jawaban tiruan.

[Hands-on P10 di repo mata kuliah](#)

Sumber: Repo informatika-itera IF25-22017, folder P10  
Integrasi AI API Hands-on.

## Bagian 2

# Memanggil Gemini dari KMP

Satu POST dengan Ktor, keluaran JSON dengan skema, dan kunci yang tidak ikut ke GitHub.

# Isi permintaan ke generateContent

## Badan JSON yang dikirim

```
{
  "contents": [{ "parts": [{ "text": "..."}] }],
  "generationConfig": {
    "responseMimeType": "application/json",
    "responseSchema": {
      "type": "OBJECT",
      "properties": {
        "kategori": { "type": "STRING" },
        "gedung": { "type": "STRING" },
        "urgensi": { "type": "STRING",
          "enum": ["Rendah", "Sedang", "Tinggi"] }
      },
      "required": ["kategori", "gedung", "urgensi"]
    }
  }
}
```

| Bagian           | Gunanya                                  |
|------------------|------------------------------------------|
| contents         | Prompt dari susunPrompt                  |
| responseMimeType | Minta jawaban berupa JSON, bukan kalimat |
| responseSchema   | Nama field, tipe, dan pilihan yang boleh |
| required         | Field yang wajib ada                     |

Nama field dan nama model selalu cocokkan dengan dokumentasi Gemini. API ini masih sering diperbarui.

# Skema ditulis sekali di Kotlin

commonMain/ai/Skema.kt

```
@Serializable data class Skema(  
    val type: String,  
    val properties: Map<String, Skema>? = null,  
    val enum: List<String>? = null,  
    val required: List<String>? = null  
)  
  
val SKEMA_SARAN = Skema("OBJECT",  
    properties = mapOf(  
        "kategori" to Skema("STRING", enum = KATEGORI),  
        "gedung" to Skema("STRING", enum = GEDUNG),  
        "urgensi" to Skema("STRING", enum = URGENSI),  
        "alasan" to Skema("STRING")  
    ),  
    required = listOf("kategori", "gedung", "urgensi")  
)
```

## Satu daftar, tiga tempat

Daftar **KATEGORI** yang sama dipakai di form, di prompt, dan di skema. Kalau kategori baru ditambah, ketiganya ikut berubah.

Field **alasan** tidak wajib. Isinya ditampilkan kecil di kartu saran.

# Jawaban menjadi data class

commonMain/ai/SaranLaporan.kt

```
@Serializable
data class SaranLaporan(
    val kategori: String,
    val gedung: String,
    val urgensi: String,
    val alasan: String = ""
)
```

JSON dari Gemini ada di dalam teks: `candidates`, `content`, `parts`, lalu `text`. Jadi kita membaca dua kali.

commonMain/ai/GeminiDto.kt

```
@Serializable data class Part(val text: String)
@Serializable data class Content(val parts: List<Part>)
@Serializable data class Candidate(
    val content: Content? = null)
@Serializable data class GeminiResponse(
    val candidates: List<Candidate>? = null)
@Serializable data class GenerationConfig(
    val responseMimeType: String,
    val responseSchema: Skema)
@Serializable data class GeminiRequest(
    val contents: List<Content>,
    val generationConfig: GenerationConfig)
```

# Kunci API tidak masuk repository

## local.properties

```
# local.properties, sudah ada di .gitignore  
GEMINI_API_KEY=isi-kunci-dari-ai-studio
```

## composeApp/build.gradle.kts

```
val lokal = Properties().apply {  
    val f = rootProject.file("local.properties")  
    if (f.exists()) f.inputStream().use { load(it) }  
}  
android {  
    buildFeatures { buildConfig = true }  
    defaultConfig {  
        buildConfigField("String", "GEMINI_API_KEY",  
            "\"${lokal.getProperty("GEMINI_API_KEY", "")}\"")  
    }  
}
```

## Di mana

## Cara membaca kunci

Android

BuildConfig.GEMINI\_API\_KEY

Desktop

System.getenv("GEMINI\_API\_KEY")

Rilis sungguhan

Server perantara milik kelompok, kunci tidak ada di aplikasi

Kunci dikirim ke **AsistenGemini** lewat konstruktor, sama seperti implementasi platform minggu lalu.

# AsistenGemini memanggil generateContent

commonMain/ai/AsistenGemini.kt

```
class AsistenGemini(  
    private val client: HttpClient,  
    private val kunci: String  
) : AsistenLaporan {  
    override suspend fun sarankan(isi: String) = runCatching {  
        val jawab: GeminiResponse = client.post(URL_GEMINI) {  
            parameter("key", kunci)  
            contentType(ContentType.Application.Json)  
            setBody(permintaan(susunPrompt(isi)))  
        }.body()  
        val teks = jawab.candidates?.firstOrNull()  
            ?.content?.parts?.firstOrNull()?.text  
            ?: error("Gemini tidak memberi jawaban")  
        Json.decodeFromString<SaranLaporan>(teks)  
    }  
}
```

| Bagian       | Artinya                                |
|--------------|----------------------------------------|
| URL_GEMINI   | BASE_URL, model, lalu :generateContent |
| permintaan() | Membungkus prompt dan SKEMA_SARAN      |
| runCatching  | Error apa pun menjadi Result.failure   |

BANGUN BARENG

# **responseMimeType dan skema kita hapus, prompt tetap minta JSON**

Lalu kita panggil `decodeFromString` pada jawabannya. Apakah selalu berhasil? Tulis tebakanmu, baru kita jalankan lima kali.

# Satu interface, dua implementasi

commonMain/ai/AsistenLaporan.kt

```
interface AsistenLaporan {  
    suspend fun sarankan(isi: String): Result<SaranLaporan>  
}  
  
class AsistenPalsu : AsistenLaporan {  
    override suspend fun sarankan(isi: String) = runCatching {  
        delay(1000)  
        SaranLaporan("AC", "GKU 1", "Sedang", "data palsu")  
    }  
}
```

| LaporKampus    | Hands-on P10     |
|----------------|------------------|
| AsistenLaporan | AiRepository     |
| AsistenPalsu   | MockAiRepository |
| AsistenGemini  | RealAiRepository |

Layar Asisten dikembangkan dengan versi palsu, tanpa kunci dan tanpa kuota. Minggu depan versi palsu ini juga dipakai di unit test.

**Bagian 3**

# AI yang menyarankan, bukan memutuskan

Keadaan layar, kartu saran yang bisa dikoreksi, saat AI gagal, dan sepuluh kasus uji.

# Empat keadaan layar Asisten

commonMain/ui/AsistenUiState.kt

```
sealed interface AsistenUiState {  
    data object Kosong : AsistenUiState  
    data object Memuat : AsistenUiState  
    data class Saran(val isi: SaranLaporan) : AsistenUiState  
    data class Gagal(val pesan: String) : AsistenUiState  
}
```

| Keadaan | Yang tampil                             |
|---------|-----------------------------------------|
| Kosong  | Kolom deskripsi dan tombol Minta saran  |
| Memuat  | Indikator memuat, tombol nonaktif       |
| Saran   | Kartu Saran AI dengan Pakai dan Abaikan |
| Gagal   | Pesan singkat, form tetap bisa diisi    |

commonMain/ui/AsistenViewModel.kt

```
class AsistenViewModel(  
    private val asisten: AsistenLaporan  
) : ViewModel() {  
    private val _ui = MutableStateFlow<AsistenUiState>(Kosong)  
    val ui = _ui.asStateFlow()  
  
    fun minta(deskripsi: String) = viewModelScope.launch {  
        _ui.value = Memuat  
        _ui.value = asisten.sarankan(deskripsi).fold(  
            onSuccess = { Saran(it) },  
            onFailure = { Gagal("Saran AI belum bisa dibuat") }  
        )  
    }  
}
```

ViewModel hanya tahu **AsistenLaporan**. Ia tidak tahu ada Gemini, Ktor, atau kunci API.

Pesan untuk pengguna dibuat pendek. Detail error dicatat ke log untuk developer.

# Kartu saran yang bisa dikoreksi

## Asisten laporan

### Saran AI

Kategori

AC

Gedung

GKU 1

Urgensi

Tinggi

Abaikan

Pakai

1

### Diberi label

Pelapor tahu isian ini dari AI, bukan dari admin.

2

### Bisa diubah

Pakai menyalin saran ke form, dan setiap kolom tetap bisa diubah.

3

### Tidak tersimpan otomatis

Laporan baru terkirim saat pelapor sendiri menekan Kirim.

# Saat AI gagal, dan yang tidak dikirim

| Keadaan                           | Yang dilakukan aplikasi                                           |
|-----------------------------------|-------------------------------------------------------------------|
| Timeout atau internet kampus mati | Pesan singkat dan tombol coba lagi, form tetap bisa diisi         |
| Kuota habis, HTTP 429             | Minta pelapor mencoba lagi nanti, jangan mengulang otomatis terus |
| Kunci salah atau ditolak          | Pesan umum ke pengguna, detail ke log                             |
| Jawaban kosong atau JSON rusak    | Anggap gagal, jangan mengisi form dengan tebakan                  |

## Jangan dikirim ke AI

NIM, nomor HP, nama orang, dan foto wajah. Kirim deskripsi kerusakan saja, dan beri tahu pengguna bahwa teksnya diproses layanan AI.

# Sepuluh kasus uji sebelum percaya

| No | Deskripsi                                    | Kategori benar | Urgensi benar |
|----|----------------------------------------------|----------------|---------------|
| 1  | AC di GKU 1 R201 netes ke meja, lantai licin | AC             | Tinggi        |
| 2  | Lampu koridor Labtek 3 berkedip              | Listrik        | Rendah        |
| 3  | Stopkontak R305 gosong                       | Listrik        | Tinggi        |
| 4  | Air hujan masuk lewat plafon                 | Bangunan       | Sedang        |

Kolom benar diisi manusia lebih dulu, sebelum melihat jawaban AI. Lalu jalankan kesepuluh kasus, hitung yang cocok, dan ulangi setiap kali prompt diubah.

# Menghitung akurasi saran AI

## Evaluasi 10 kasus uji: kategori

| No | Deskripsi                       | Benar      | Saran AI   | Hasil |
|----|---------------------------------|------------|------------|-------|
| 1  | AC di GKU 1 R201 netes ke meja  | AC         | AC         | COCOK |
| 2  | Lampu koridor Labtek 3 berkedip | Listrik    | Listrik    | COCOK |
| 3  | Wastafel toilet GKU 2 mampet    | Air        | Air        | COCOK |
| 4  | Sampah menumpuk di depan kantin | Kebersihan | Kebersihan | COCOK |
| 5  | Stopkontak R305 gosong          | Listrik    | Listrik    | COCOK |
| 6  | Proyektor R201 tidak mau nyala  | Lainnya    | Listrik    | SALAH |
| 7  | Keran tempat wudu bocor         | Air        | Air        | COCOK |
| 8  | Kaki kursi kelas patah          | Bangunan   | Bangunan   | COCOK |
| 9  | AC lab berisik dan tidak dingin | AC         | AC         | COCOK |
| 10 | Air hujan masuk lewat plafon    | Bangunan   | Air        | SALAH |

Akurasi adalah jumlah cocok dibagi jumlah kasus. Ukur sebelum dan sesudah prompt diubah.

### Akurasi kategori

# 80%

8 cocok dari 10

Dua yang salah  
jadi bahan perbaikan  
prompt

## COBA LANGSUNG



Scan atau klik, tekan Run. Lalu ubah satu saran AI yang salah menjadi benar, dan lihat akurasi naik.

[Hitung akurasi di Kotlin Playground](#)

Sumber: Kotlin Playground, JetBrains. Data kategori sama dengan tabel di animasi.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

#### Bagian 4

# Coba sendiri, lalu bawa ke proyekmu

Tiga latihan AI API dengan observasi praktik, fitur AI tema kelompok, checkpoint proyek, dan Laporan hasil proyek.

# Yang saya amati selama latihan hari ini

Indikator RPS: **Fitur AI berfungsi dalam aplikasi.** Diamati dari latihan 1 sampai 3, AiRepository, prompt, dan layar chat.

| Kriteria                                  | Sangat baik<br>85 sampai 100                                                         | Baik<br>70 sampai 84                                    | Cukup<br>55 sampai 69                         | Kurang<br>di bawah 55 |
|-------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------|-----------------------|
| <b>Fitur AI berfungsi</b><br>CPMK0503, 4% | Layar chat memanggil repository AI, prompt memakai instruksi sistem, error ditangani | Fitur jalan, prompt atau penanganan error belum lengkap | Fitur jalan dengan kunci API tertulis di kode | Belum memanggil AI    |

Individu, bobot 4% nilai akhir. Bukti: push ke repository latihan pribadimu, lalu kirim tautan commit di LMS sebelum kelas selesai.

# Tiga latihan di repo mata kuliah

## Latihan 1

Handson1RealAiRepository.kt  
Lengkapi ask(): badan permintaan,  
POST ke generateContent, parsing,  
dan Result.

## Latihan 2

Handson2PromptEngineering.kt  
buildPrompt() menaruh instruksi  
sistem di depan pertanyaan  
pengguna.

## Latihan 3

Handson3ChatScreen.kt  
Layar chat dengan daftar pesan,  
tombol kirim, dan indikator memuat.

Semuanya di folder **P10 Integrasi AI API Hands-on** dan jalan tanpa kunci karena memakai MockAiRepository. Yang punya kunci: simpan hanya di salinan lokal, cocokkan GEMINI\_MODEL dengan dokumentasi, lalu tambahkan responseType.

# Kalau error

| Gejala atau pesan                  | Penyebab                                   | Yang dicoba                                   |
|------------------------------------|--------------------------------------------|-----------------------------------------------|
| Jawaban selalu Error: di latihan 1 | Kunci masih YOUR_API_KEY_HERE              | Isi kunci asli dari AI Studio                 |
| HTTP 400 Bad Request               | Bentuk JSON permintaan tidak sesuai        | Cocokkan data class dengan dokumentasi Gemini |
| HTTP 403 atau kunci tidak valid    | Kunci salah, dihapus, atau kena batas      | Buat kunci baru, cek kuota harian             |
| HTTP 404 di generateContent        | Nama model salah atau sudah tidak dilayani | Ganti GEMINI_MODEL sesuai dokumentasi         |
| candidates kosong                  | Prompt kena filter keamanan                | Kembalikan Result.failure dengan pesan jelas  |
| SerializationException             | Jawaban bukan JSON atau nama field berbeda | Pasang responseType dan skema                 |

# Fitur AI untuk tema kelompokmu

**1** **Prompt tema**  
Peran, konteks, format, dan contoh untuk entitas utama tema kalian.

**2** **Skema dan data class**  
Keluaran JSON terstruktur yang langsung di-parse.

**3** **Saran yang bisa dikoreksi**  
Label Saran AI, bisa diubah, tidak tersimpan otomatis.

**4** **Sepuluh kasus uji**  
Tabel kasus dan akurasi di README, diperbarui setiap prompt berubah.

## CONTOH DARI LAPORKAMPUS

Deskripsi menjadi saran kategori, gedung, dan urgensi di tab Asisten. Pelapor menekan Pakai, mengubah urgensi, lalu mengirim sendiri.

# Checkpoint proyek minggu 9

- 1 Fitur AI tema**  
Satu panggilan Gemini dengan keluaran JSON sesuai skema.
- 2 Bisa dikoreksi**  
Hasil tampil sebagai saran yang bisa diubah atau diabaikan.
- 3 Kunci aman**  
Kunci di local.properties, tidak ada di riwayat commit.

## Dikumpulkan

Push ke repository kelompok. README memuat prompt, skema, dan tangkapan layar kartu saran AI.

**Dicek di awal pertemuan berikutnya: tuntas, sebagian, atau belum.**

# Rubrik laporan hasil proyek

| Kriteria                                            | Sangat baik<br>85 sampai 100                                             | Baik<br>70 sampai 84              | Cukup<br>55 sampai 69 | Kurang<br>di bawah 55 |
|-----------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------|-----------------------|-----------------------|
| <b>Laporan fitur AI</b><br>CPMK0503, 5%<br>Kelompok | Prompt, skema keluaran,<br>evaluasi 10 kasus uji, dan<br>batasan dibahas | Lengkap tanpa evaluasi<br>terukur | Hanya deskripsi fitur | Tidak ada laporan     |

Laporan Hasil Proyek, bobot 5% nilai akhir: CPMK0503 5%. Nilai kriteria dikalikan bobotnya. Plagiat bernilai 0.

TUTUP

# Tiket keluar



**1** Kunci  
Kenapa kunci API tidak boleh ada di repository?

**2** Skema  
Apa gunanya meminta keluaran JSON dengan skema?

**3** Saran  
Kenapa hasil AI ditampilkan sebagai saran yang bisa dikoreksi?

Jawab singkat di kertas atau formulir LMS sebelum keluar kelas.

TUTUP

# Minggu depan: testing dan dependency injection



## MASALAH MINGGU 10

Setiap kali menambah fitur, ada fitur lama yang diam-diam rusak dan baru ketahuan saat demo. Bagaimana memastikan kode tetap benar tanpa mengetes manual satu per satu?

## Persiapan

Bawa fitur AI kelompok dan AsistenPalsu kalian. Minggu depan keduanya menjadi bahan unit test pertama.

# Terima kasih

Sampai jumpa di pertemuan 10

---

Ada pertanyaan? Sampaikan sekarang, lewat LMS, atau email ke [muhammad.algifari@if.itera.ac.id](mailto:muhammad.algifari@if.itera.ac.id)